LAPORAN PRATIKUM

STRUKTUR DATA

ABSTRACT DATA TYPE JAM

disusun Oleh:

Annisa Layli Ramadhani

2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T

Asisten Pratikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2025

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salaj satu bentuk pemenuhan tugas praktikum serta untuk memahami penerapan konsep Abstract Data Type (ADT) dalam bahasa java.

Dalam laporan ini dibahas implementasi kelas jam yang mencakup berbagai operasi seperti validasi, konstruktor, selektor, mutator, serta operasi konversi dan aritmatika waktu. Melalui praktikum ini, diharapkan penulis dapat lebih memahami konsep pemrograman berorientasi objek secara lebih mendalam.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 31 Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 5
BAB II PEMBAHASAN 6
2.1 Kode Program Praktikum Pekan 1 6
2.1.1 Jam_2511532024 6
2.1.2 JamDriver_2511532024 9
2.1.3 JamDriver2_2511532024 10
2.2 Tugas Praktikum Pekan 1 12
2.2.1 Soal Tugas Praktikum Pekan 1 12
2.2.2 Kode Program Mobil_2511532024 12
2.2.3 Penjelasan Kode Program Mobil_2511532024 14
2.2.4 Kode Program MobilDriver_2511532024 17
2.2.5 Penjelasan Kode Program MobilDriver_2511532024 18
2.2.6 Output Kode Program 21
BAB III PENUTUP 22
3.1 Kesimpulan 22
DAFTAR PUSTAKA 23

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut pemahaman yang baik terhadap konsep dasar pemrograman, khususnya dalam pengelolaan data secara terstruktur. Salah satu konsep penting dalam ilmu komputer adalah ADT, yang memungkinkan suatu data dipresentasikan bersama dengan operasi-operasi yang dapat dilakukan terhadap data tersebut.

Dalam pemrograman berorientasi objek, ADT dapat diimplementasikan melalui pembuatan kelas yang mencerminkan suatu objek nyata. Salah satu contoh yang sederhana namun penting adalah representasi waktu dalam bentuk jam, menit, dan detik. Pengelolaan data waktu waktu sering digunakan dalam berbagai aplikasi, sehingga diperlukan struktur yang jelas dan operasi yang tepat untuk memanipulasinya.

Oleh karena itu, pada praktikum ini dilakukan implementasi ADT jam menggunakan bahasa java. Melalui pembuatan kelas ini, diharapkan mahasiswa dapat memahami bagaimana cara mendefinisikan atribut, membuat konstruktor, serta mengimplementasikan berbagai operasi seperti validasi, konversi, dan perhitungan waktu secara sistematis dan terstruktur.

1.2 Tujuan

Tujuan dari pelakasanaan praktikum antara lain sebagai berikut:

1. Memahami konsep ADT dalam pemrograman.
2. Menerapkan konsep ADT ke dalam bahasa java melalui pembuatan kelas jam.
3. Memahami penggunaan konstruktor, selektor, mutator, serta operasi-operasi pada objek.

4. Melatih kemampuan dalam mengelola dan memanipulasi data waktu secara terstruktur.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Meningkatkan pemahaman tentang pemrogram berorientasi objek.
2. Membantu dalam memahami cara kerja representasi data waktu dalam program.
3. Menjadi dasar dalam pengembangan program yang lebih kompleks.
4. Melatih kemampuan berpikir logis dan sistematis dalam pemrograman.

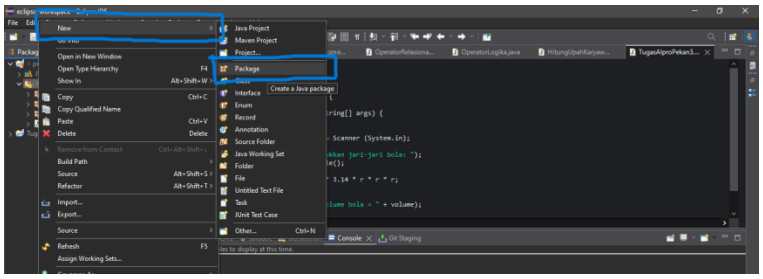
BAB II

PEMBAHASAN

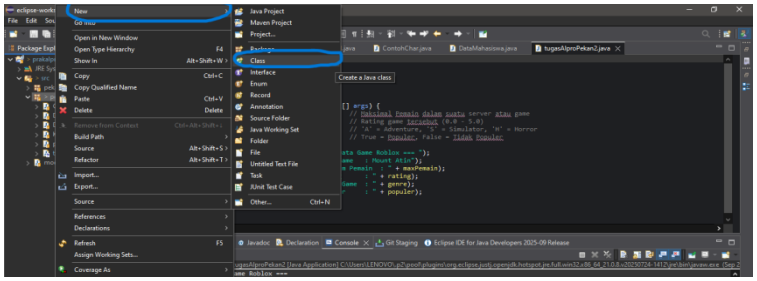
2.1 Kode Program Praktikum Pekan 1

2.1.1 Jam_2511532024

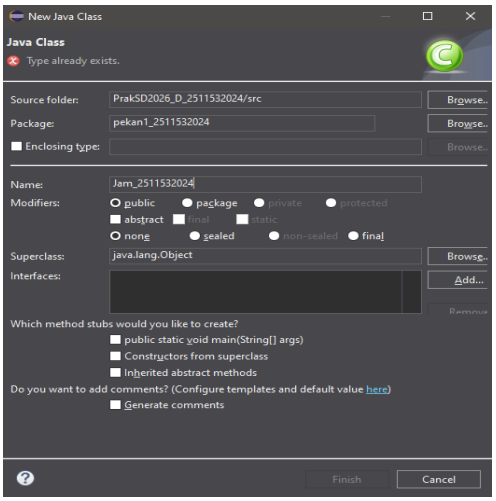
1. Buka aplikasi eclipse, lalu buat package baru pada bagian src klik kanan, lalu pilih “New” dan klik tulisan package.



2. Kemudian buat nama packagenya tanpa spasi, huruf kapital, karakter khusus. Beri nama “pekan1_2511532024”. Lalu klik finish.
3. Ketika package sudah jadi. Klik pada bagian “pekan1_2511532024”. Lalu klik “New” dan pilih bagian “Class” untuk memulai membuat program.



4. Buat nama Class program yang akan dibuat pada bagian nama tanpa spasi dan menggunakan huruf kapital pada awal kata, lalu klik bagian “public static void main(string[]args)”. Kemudian klik “finish”.



5. Masukkan syntax berikut.

```
1. public final class Jam_2511532024 {
2.             private int hh; // 0..23
3.             private int mm; // 0..59
4.             private int ss; // 0..59
5.
6.             // --- Validator ---
7.             public static boolean isValid(int h,
   int m, int s) {
8.                     return (0 <= h && h <= 23) && (0
   <= m && m <= 59) && (0 <= s && s <= 59);
9.             }
10.
11.            // --- Konstruktor/Kreator ---
12.            public Jam_2511532024(int h, int m, int
   s) {
13.                    this.hh = h; this.mm = m;
   this.ss = s;
14.            }
15.
16.            // --- Selektor ---
17.            public int getHour() {
18.                    return hh;
19.            }
20.            public int getMinute() {
21.                    return mm;
22.            }
23.            public int getSecond() {
24.                    return ss;
25.            }
26.
27.            // --- Mutator ---
28.            public void setHour(int h) {
29.                    this.hh = h;
30.            }
31.            public void setMinute(int m) {
32.                    this.mm = m;
33.            }
34.            public void setSecond(int s) {
35.                    this.ss = s;
```

```java
        }

        // --- Konversi ---
        public int toSeconds( ) { return hh * 3600 + mm * 60 + ss;}
        public static Jam_2511532024 fromSeconds(int total) {
            if (total < 0) throw new IllegalArgumentException("detik negatif");
            total %= 24 * 3600; // wrap 24 jam
            int h = total / 3600;
            total %= 3600;
            int m = total / 60;
            int s = total % 60;
            return new Jam_2511532024(h,m,s);
        }

        // --- Relasional ---
        public int compareTo(Jam_2511532024 other) {
            return Integer.compare(this.toSeconds(), other.toSeconds());
        }
        public boolean equals(Object o) {
            if (!(o instanceof Jam_2511532024 j)) return false;
            return hh == j.hh && mm == j.mm && ss == j.ss;
        }
        public int hashCode() { return java.util.Objects.hash(hh, mm, ss);}

        // --- Aritmatika ---
        public Jam_2511532024 plus(Jam_2511532024 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
        public Jam_2511532024 minus(Jam_2511532024 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
        public Jam_2511532024 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
        public Jam_2511532024 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
        public Jam_2511532024 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
        public Jam_2511532024 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }

```

```
68.             // --- Durasi (detik) -- bisa neagtif
    jika this > other sesuai spesifikasi durasi(JAW,
    JAkh)
69.             public static int
    durasiDetik(Jam_2511532024 jaw, Jam_2511532024
    jakh) { return jakh.toSeconds() -
    jaw.toSeconds(); }
70.             // overriding ke string
71.             public String toString() { return
    String.format("%02d:%02d:%02d", hh, mm, ss); }
72.     }
```

2.1.2 JamDriver_2511532024

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut "JamDriver_2511532024". Lalu masukkan syntax seperti berikut.

```
1.  public class JamDriver_2511532024 {
2.
3.      public static void main(String[] args) {
4.              Jam_2511532024 a = new Jam_2511532024
    (23, 59, 50);
5.              Jam_2511532024 b = new Jam_2511532024
    (0, 0, 15);
6.              System.out.println("a              = "
    + a);
7.              System.out.println("b              = "
    + b);
8.              System.out.println("a+b            = "
    + a.plus(b));
9.              System.out.println("next 20s       = "
    + a.nextNSeconds(20));
10.             System.out.println("durasi(a,b)    = "
    + Jam_2511532024.durasiDetik(a, b));
11.             System.out.println("a.compareTo(b) = "
    + a.compareTo(b));
12.     }
13. }
```

2. Klik "run" untuk melihat hasil syntax yang sudah dibuat.

```
<terminated> JamDriver_2511532024 [Java A
a               = 23:59:50
b               = 00:00:15
a+b             = 00:00:05
next 20s        = 00:00:10
durasi(a,b)     = -86375
a.compareTo(b)  = 1
```

2.1.3 JamDriver2_2511532024

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “Nilai”. Lalu masukkan syntax seperti berikut.

```
import java.util.Scanner;

public class JamDriver2_2511532024 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek JAM ===");
        // 1. Input Jam pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511532024 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511532024 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)          : " + j1.toString());
        System.out.println("Jam 2 (String)          : " + j2.toString());
        System.out.println("Jam 1 dalam detik       : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik       : " + j2.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                  : Jam 1 lebih lambat (setelah) Jam 2");
        }
        else if (perbandingan < 0) {
            System.out.println("Status                  : Jam 1 lebih awal (sebelum) Jam 2");
        }
        else {
            System.out.println("Status                  : Jam 1 dan Jam 2 sama persis");
        }

```

```java
            // 5. Operasi Aritmatika
            System.out.println("Durasi (J1 ke J2)         : " + Jam_2511532024.durasiDetik(j1, j2) + " detik");

            Jam_2511532024 jNext = j1.nextSecond();
            System.out.println("Jam 1 Detik Berikutnya    : " + jNext);

            Jam_2511532024 jPrev = j1.prevSecond();
            System.out.println("Jam 1 Detik Sebelumnya    : " + jPrev);

            // 6. Operasi Penjumlahan Jam
           Jam_2511532024 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil J1 + J2             : " + jHasilPlus);

            input.close();
           System.out.println("\nProgram  Selesai");

     }
            /**
             * Prosedur pembantu untuk melakukan input dan validasi secara berulang
             * sampai user memasukkan angka yang benar (0..23, 0..59)
             */

     private static Jam_2511532024 buatJamDariInput(Scanner sc) {
            int h, m, s;
            while (true) {
                   System.out.print("Masukkan Jam (0-23)       : ");
                   h = sc.nextInt();
                   System.out.print("Masukkan Menit (0-59)     : ");
                   m = sc.nextInt();
                   System.out.print("Masukkan Detik (0-59)     : ");
                   s = sc.nextInt();

                   // Menampilkan method static isValid dari keals Jam
                   if (Jam_2511532024.isValid(h, m, s)) {
                          return new Jam_2511532024(h, m, s);
                   } else {
                          System.out.println("[Error] Input tidak valid! Silakan ulangi. \n");
                   }
            }
     }
}
```

2. Klik “run” untuk melihat hasil syntax yang sudah dibuat.

```
=== Program Driver Objek JAM ===

--- Input Jam 1 ---
Masukkan Jam (0-23)      : 09
Masukkan Menit (0-59)    : 10
Masukkan Detik (0-59)    : 06

--- Input Jam 2 ---
Masukkan Jam (0-23)      : 11
Masukkan Menit (0-59)    : 04
Masukkan Detik (0-59)    : 00

--- Hasil Operasi ---
Jam 1 (String)           : 09:10:06
Jam 2 (String)           : 11:04:00
Jam 1 dalam detik        : 33006
Jam 2 dalam detik        : 39840
Status                   : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)        : 6834 detik
Jam 1 Detik Berikutnya   : 09:10:07
Jam 1 Detik Sebelumnya   : 09:10:05
Hasil J1 + J2            : 20:14:06

Program Selesai
```

2.2 Tugas Praktikum Pekan 1

2.2.1 Soal Tugas Praktikum Pekan 1

Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk. Contoh : Avanza, 2020, 1300, 150000000, Toyota. Kemudian buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, selektor, dan mutator. Dan buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim.

2.2.2 Kode Program Mobil_2511532024

```
1. public final class Mobil_2511532024{
2.  private String nama;
3.  private int tahun;
4.  private int cc;
5.  private double harga;
6.  private String merk;
7.
8.  // --- Validator ---
9. public static boolean isValid(String nama, int tahun,
   int cc, double harga, String merk) {
10.        return (nama != null && !nama.isEmpty())
```

```
                    && (tahun > 1900)
                    && (cc > 0)
                    && (harga >= 0)
                    && (merk != null && !merk.isEmpty());
}

// --- Konstruktor ---
public Mobil_2511532024(String nama, int tahun, int cc, double harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
}

// --- Selektor ---
public String getNama() { return nama; }
public int getTahun() { return tahun; }
public int getCc() { return cc; }
public double getHarga() { return harga; }
public String getMerk() { return merk; }

// --- Mutator ---
public void setNama(String nama) { this.nama = nama; }
public void setTahun(int tahun) { this.tahun = tahun; }
public void setCc(int cc) { this.cc = cc; }
public void setHarga(double harga) { this.harga = harga; }
public void setMerk(String merk) { this.merk = merk; }

// --- Tambah Mobil ---
public static int tambahMobil(Mobil_2511532024[] arr, int size, Mobil_2511532024 m) {
        if (size < arr.length) {
                arr[size] = m;
                return size + 1;
        } else {
                System.out.println("Array penuh!");
                return size;
        }
}

// --- Hapus Mobil ---
public static int hapusMobil(Mobil_2511532024[] arr, int size, String nama) {
        for (int i = 0; i < size; i++) {
                if (arr[i].getNama().equalsIgnoreCase(nama)) {
                        for (int j = i; j < size - 1; j++) {
                                arr[j] = arr[j + 1];
                        }
```

```
                    arr[size - 1] = null;
                    return size - 1;
                }
        }
        return size;
}

// --- Output ---
public String toString() {
        return String.format("%s %d %dcc Rp%.0f %s",
    nama, tahun, cc, harga, merk);
}
```

2.2.3 Penjelasan Kode Program Mobil_2511532024

1. Atribut

```
private String nama;
private int tahun;
private int cc;
private double harga;
private String merk;
```

Bagian ini merupakan atribut dari class Mobil yang digunakan untuk menyimpan data mobil, yaitu nama, tahun produksi, kapasitas mesin, harga, dan merk. Semua atribut dibuat **private** agar tidak bisa diakses langsung dari luar class, sehingga harus menggunakan method tertentu untuk mengaksesnya.

2. Validator

```
public static boolean isValid(String nama, int tahun, int cc,
double harga, String merk)
```

Bagian ini merupakan method validator yang berfungsi untuk mengecek apakah data mobil yang dimasukkan valid atau tidak. Method ini akan mengembalikan nilai true jika semua data sesuai syarat.

```
Return (nama != null && !nama.isEmpty())
    && (tahun > 1900)
    && (cc > 0)
```

```
    && (harga >= 0)
    && (merk != null && !merk.isEmpty());
```

Baris ini berisi kondisi-kondisi yang harus dipenuhi agar data dianggap valid. Semua kondisi digabung menggunakan operator **&&**, sehingga seluruh syarat harus bernilai benar.

3. Konstruktor

```
Public Mobil_2511532024(String nama, int tahun, int cc, double harga, String merk)
```

Bagian ini merupakan konstruktor yang digunakan untuk membuat objek mobil dengan mengisi nilai atribut sesuai parameter yang diberikan.

```
this.nama = nama;
```

Kata kunci **this** digunakan untuk membedakan antara atribut class dengan parameter yang diterima oleh konstruktor.

4. Selektor (Getter)

```
public String getNama() { return nama; }
```

Bagian ini merupakan method getter yang berfungsi untuk mengambil nilai atribut dari objek mobil tanpa mengaksesnya secara langsung.

5. Mutator (Setter)

```
public void setNama(String nama) { this.nama = nama; }
```

Bagian ini merupakan method setter yang digunakan untuk mengubah nilai atribut pada objek mobil.

6. Method Tambah Mobil

```
public static int tambahMobil(Mobil_2511532024[] arr, int size, Mobil_2511532024 m)
```

Bagian ini merupakan method untuk menambahkan objek mobil ke dalam array. Method ini menggunakan parameter array dan ukuran data saat ini.

```
if (size < arr.length)
```

Baris ini digunakan untuk mengecek apakah array masih memiliki ruang kosong.

```
arr[size] = m;
return size + 1;
```

Bagian ini digunakan untuk menyimpan objek mobil ke dalam array pada posisi terakhir, kemudian ukuran data ditambah satu.

7. Method Hapus Mobil

```
public static int hapusMobil(Mobil_2511532024[] arr, int size, String nama)
```

Bagian ini merupakan method untuk menghapus data mobil berdasarkan nama yang dicari dalam array.

```
arr[j] = arr [j + 1];
```

Baris ini untuk menggeser elemen array ke kiri setelah data yang dihapus ditemukan, agar tidak ada ruang kosong di tengah array.

```
arr[size – 1] = null;
return size – 1;
```

Bagian ini digunakan untuk menghapus elemen terakhir setelah proses penggeseran dan mengurangi jumlah data.

8. Method Output

```
public String toString()
```

Bagian ini merupakan method yang digunakan untuk mengubah objek mobil menjadi bentuk string agar dapat ditampilkan.

```
return String.format("%s %d %dcc Rp%.0f %s", nama,
```

tahun, cc, harga, merk);

Baris ini digunakan untuk menyusun tampilan data mobil menjadi format yang rapi dan mudah dibaca.

2.2.4 Kode Program MobilDriver_2511532024

```
1.  public class MobilDriver_2511532024 {
2.
3.   public static void tampilkanData(Mobil_2511532024[]
    arr, int size) {
4.
    System.out.println("================================
    ===========");
5.          System.out.println("  Nama   |Tahun |  CC   |
    Harga   |  Merk");
6.
    System.out.println("================================
    ===========");
7.
8.          for (int i = 0; i < size; i++) {
9.                System.out.println(
10.                             arr[i].getNama() + " | " +
11.                             arr[i].getTahun() + " | "
    +
12.                             arr[i].getCc() + " | Rp" +
13.                             arr[i].getHarga() + " | "
    +
14.                             arr[i].getMerk()
15.               );
16.         }
17.
18.
    System.out.println("================================
    ===========");
19. }
20.
21. public static void main(String[] args) {
22.
23.         Mobil_2511532024[] daftar = new
    Mobil_2511532024[10];
24.         int size = 0;
25.
26.         Mobil_2511532024 m1 = new
    Mobil_2511532024("911 GT3", 2022, 3996, 2500000000.0,
    "Porsche");
27.         Mobil_2511532024 m2 = new
    Mobil_2511532024("488 GTB", 2022, 3900, 5000000000.0,
    "Ferrari");
28.
29.         // tambah
30.         size = Mobil_2511532024.tambahMobil(daftar,
    size, m1);
31.         size = Mobil_2511532024.tambahMobil(daftar,
    size, m2);
```

```java

        System.out.println("\n=== DAFTAR MOBIL ===");
        tampilkanData(daftar, size);

        // update
        daftar[0].setHarga(2600000000.0);

        System.out.println("\n=== SETELAH UPDATE ===");
        tampilkanData(daftar, size);

        // hapus
        size = Mobil_2511532024.hapusMobil(daftar, size, "488 GTB");

       System.out.println("\n=== SETELAH HAPUS ===");
        tampilkanData(daftar, size);
 }

}
```

2.2.5 Penjelasan Kode Program MobilDriver_2511532024

1. Deklarasi Class Driver

```
public class Mobil_2511532024 {
```

Bagian ini merupakan class driver yang berfungsi sebagai program utama untuk menjalankan dan menguji ADT Mobil. Di sinilah semua method dari class Mobil akan digunakan.

2. Method tampilanData

```
public static void tampilkanData(Mobil_2511532024[] arr, int size)
```

Bagian ini merupakan method tambahan yang digunakan untuk menampilkan mobil dalam array. Method ini menerima parameter array mobil dan jumlah data yang tersimpan.

```
System.out.println("==========================");
System.out.println("Nama | Tahun | CC | Harga | Merk");
System.out.println("==========================");
```

Baris ini digunakan untuk membuat tampilan header agar output terlihat lebih rapi dan terstruktur.

```
for (int i = 0, i < size; i++) {
```

Perulangan ini digunakan untuk menampilkan setiap data mobil dalam array sesuai jumlah data yang ada.

```
arr[i].getNama()
```

Bagian ini memanggil method getter untuk mengambil nilai atribut dari objek mobil, seperti nama, tahun, cc, harga, dan merk.

```
System.out.println(...);
```

Baris ini digunakan untuk menampilkan data mobil dalam satu baris dengan format sederhana.

3. Method Main

```
public static void main(String[] args) {
```

Bagian ini merupakan method utama yang akan dijalankan pertama kali saat program dieksekusi.

4. Deklarasi Array

```
Mobil_2511532024[] daftar = new Mobil_2511532024[10];
int size = 0;
```

Baris ini digunakan untuk membuat array yang dapat menampung maksimal 10 data mobil, serta variabel **size** untuk menyimpan jumlah data yag sudah terisi.

5. Membuat Objek Mobil

```
Mobil_2511532024 m1 = new Mobil_2511532024("911 GT3", 2022, 3996, 2500000000.0, "Porsche");
Mobil_2511532024 m2 = new Mobil_2511532024("488 GTB", 2022, 3900, 5000000000.0, "Ferrari");
```

Bagian ini digunakan untuk membuat dua objek mobil dengan data yang berbeda.

6. Menambah Data Mobil

```
size = Mobil_2511532024.tambahMobil(daftar, size, m1);
size = Mobil_2511532024.tambahMobil(daftar, size, m2);
```

Baris ini memanggil method **tambahMobil** untuk memasukkan obbjek mobil ke dalam array. Nilai **size** diperbarui setiap kali data ditambahkan.

7. Menampilkan Data

```
System.out.println("\n=== DAFTAR MOBIL ===");
tampilkanData(daftar, size);
```

Bagian ini digunakan untuk menampilkan seluruh data mobil yang ada di dalam array dengan meamanggil method **tampilkanData**.

8. Mengubah Data (Mutator)

```
daftar[0].setHarga(2600000000.0);
```

Baris ini digunakan untuk mengubah harga mobil pertama menggunakan method setter.

9. Menampilkan Setelah Update

```
System.out.println("\n=== SETELAH UPDATE ===");
tampilkanData(daftar, size);
```

Bagian ini menampilkan data setelah dilakukan perubahan harga.

10. Menghapus Data

```
size = Mobil_2511532024.hapusMobil(daftar, size, "488 GTB");
```

Baris ini digunakan untuk menghapus mobil dengan nama “488 GTB” dari array, kemudian ukuran data dierbarui.

11. Menampilkan Setelah Hapus

```
System.out.println("\n=== SETELAH HAPUS ===");
```

```
tampilkanData(daftar, size);
```

Bagian ini menampilkan data mobil setelah salah satu data dihapus.

2.2.6 Output Kode Program

```
=== DAFTAR MOBIL ===
==========================================
  Nama   |Tahun |  CC   |  Harga   |  Merk
==========================================
911 GT3 | 2022 | 3996 | Rp2.5E9 | Porsche
488 GTB | 2022 | 3900 | Rp5.0E9 | Ferrari
==========================================

=== SETELAH UPDATE ===
==========================================
  Nama   |Tahun |  CC   |  Harga   |  Merk
==========================================
911 GT3 | 2022 | 3996 | Rp2.6E9 | Porsche
488 GTB | 2022 | 3900 | Rp5.0E9 | Ferrari
==========================================

=== SETELAH HAPUS ===
==========================================
  Nama   |Tahun |  CC   |  Harga   |  Merk
==========================================
911 GT3 | 2022 | 3996 | Rp2.6E9 | Porsche
==========================================
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep ADT dapat diimplementasikan dengan baik menggunakan bahasa pemrograman Java melalui pembuatan kelas Jam. Kelas ini mampu merepresentasikan data waktu secara terstruktur dengan atribut jam, menit, dan detik serta dilengkapi dengan berbagai operasi seperti validasi, konversi, perbandingan, dan aritmatika waktu.

Penggunaan metode konversi ke detik menjadi inti dari proses perhitungan, sehingga operasi yang dilakukan menjadi lebih sederhana dan efisien. Selain itu, penerapan konsep enkapsulasi melalui penggunaan atribut private serta method getter dan setter membantu menjaga keamanan dan konsisten data.

Dengan demikian, praktikum ini memberikan pemahaman yang lebih baik mengenai penerapan ADT dan pemrograman berorientasi objek, serta melatih kemampuan dalam merancang program yang sistematis dan terstruktur.

DAFTAR PUSTAKA

[1] Oracle, “Java Platform, Standard Edition Documentation,” [Online]. Available: https://docs.oracle.com/javase/8/docs/. [Diakses: 31 Maret 2026].